

Enumerate Subsets

Given a set of integers A , print out all the subsets of A .

Method 1: recursively add a new member to all the current subsets

Method 2: use dfs to add a new member to the current path

Method 3: use binary numbers as bit masks

If A has n terms, then we can loop x from 0 to $2^n - 1$, then look at the binary presentation of x , if the k_{th} bit is on, then the k_{th} item is in the subset.

Method 1:

```
#include <iostream>
#include <vector>
typedef long long ll;
using namespace std;
typedef pair<int, int> pii;
int main()
{
    vector<int> x = {1, 2, 3, 4};
    vector<vector<int>> subsets(1); // start with an empty subset
    for (auto k : x)
    {
        // add element k to each existing subset
        int sz = subsets.size();
        for (int i = 0; i < sz; i++)
        {
            vector<int> clone = subsets[i];
            clone.push_back(k);
            subsets.push_back(clone);
        }
    }
    for (auto s : subsets)
    {
        for (auto a : s)
            cout << a << " ";
        cout << endl;
    }
}
```

Method 2:

Use DFS to build one element at a time

```
#include <iostream>
#include <vector>
```

```

typedef long long ll;
using namespace std;
typedef pair<int, int> pii;
int subset[10];
vector<int> x = {1, 2, 3, 4};
int counts = 0;
void print(int k)
{
    cout << ++counts << ": ";
    for (int i = 0; i < k; i++)
    {
        cout << subset[i] << " ";
    }
    cout << endl;
}
void dfs(int i) // we are building element #i in the subset
{
    print(i);
    if (i == x.size())
    {
        return;
    }
    for (int j = 0; j < x.size(); j++)
    {
        bool ok = true;
        for (int k = 0; k < i; k++)
        {
            if (subset[k] >= x[j])
                ok = false;
        }
        if (ok)
        {
            subset[i] = x[j];
            dfs(i + 1);
        }
    }
}
int main()
{
    dfs(0);
}

```

Method 3: user bit masks

```

#include <iostream>
#include <vector>
using namespace std;
// Function to print a subset based on the bitmask
void printSubset(const vector<int>& set, int bitmask) {

```

```

    cout << "{ ";
    for (int i = 0; i < set.size(); ++i) {
        // Check if the i-th bit is set in the bitmask
        if (bitmask & (1 << i)) {
            cout << set[i] << " ";
        }
    }
    cout << "}" << endl;
}

// Function to enumerate all subsets of a given set
void enumerateSubsets(const vector<int>& set) {
    int n = set.size();
    int totalSubsets = 1 << n; // 2^n subsets
    for (int bitmask = 0; bitmask < totalSubsets; ++bitmask) {
        printSubset(set, bitmask);
    }
}

int main() {
    // Define a set of elements
    vector<int> set = {1, 2, 3};
    // Enumerate all subsets of the set
    enumerateSubsets(set);
    return 0;
}

```

Bit Manipulations and Operations Guide

Friday, September 27, 2024 4:21 PM

Introduction

Bit manipulations are useful for simplifying problems and improving efficiency. They are a kind of low-level optimization tool. Today, we'll explore various bitwise operations and see how they can be used for efficient coding. We'll also look at how to print all possible subsets of a set and solve the subset sum problem.

Basic Bitwise Operations

1. Bitwise AND

$0 \ \& \ 0 = 0$

$0 \ \& \ 1 = 0$

$1 \ \& \ 0 = 0$

$1 \ \& \ 1 = 1$

Explanation: The result is 1 only if both bits are 1.

Example: 5 & 6

5 -> 101

6 -> 110

AND:

101

&

110

===

100 -> 4

Result: 4

2. Bitwise OR

$0 \ | \ 0 = 0$

$0 \ | \ 1 = 1$

$1 \ | \ 0 = 1$

$1 \ | \ 1 = 1$

Explanation: The result is 1 if any of the bits are 1.

3. Bitwise XOR : exclusive or

$$0 \wedge 0 = 0$$

$$0 \wedge 1 = 1$$

$$1 \wedge 0 = 1$$

$$1 \wedge 1 = 0$$

Explanation: The result is 1 if the bits are different.

Left Shifting

Shifts the bit-pattern of a number to the left by the specified positions.

Example: $5 \ll 1 \rightarrow 101 \ll 1 \rightarrow 1010$ (10 in decimal)

Right Shifting

Shifts the bit-pattern of a number to the right by the specified positions.

Example: $5 \gg 1 \rightarrow 101 \gg 1 \rightarrow 10$ (2 in decimal)

Bit Masking

Masking: The concept of hiding bits.

1. OR with 1: $\text{bit} | 1$ always results in 1.
2. AND with 0: $\text{bit} \& 0$ always results in 0.

Useful Bitwise Operations

1. Multiply by 2

Left shift the number by 1: $n \ll 1$.

Example: $3 \ll 1 = 6$.

2. Divide by 2

Right shift the number by 1: $n \gg 1$.

Example: $4 \gg 1 = 2$.

3. Finding Powers of 2

Use left shift: $1 \ll \text{exponent}$.

Example: $1 \ll 3 = 8$.

4. Check if a Number is a Power of 2

Use the expression: $n \& (n - 1) == 0$.

Example: For $n = 8$ (1000 in binary), $8 \& (8 - 1) = 1000 \& 0111 = 0$.

5. Convert Decimal to Binary

Loop and print: $\text{number} \& 1$, then right shift the number.

Example: For 5: 101 $\rightarrow$ 1 0 1.

6. Check if a Number is Even or Odd

Use: $\text{number} \& 1$.

Example: $5 \& 1 = 1$ (Odd), $4 \& 1 = 0$ (Even).

7. Get the Status of the x-th Bit

Use: $\text{number} \& (1 \ll (x - 1))$.

Example: For $n = 5$ (101 in binary), to check 3rd bit: $5 \& (1 \ll 2) = 101 \& 100 = 4$ (SET).

8. Set the x-th Bit

Use: $\text{number} \mid (1 \ll (x - 1))$.

Example: For $n = 5$ (101 in binary), to set 2nd bit: $5 \mid (1 \ll 1) = 101 \mid 010 = 7$.

9. Toggle the x-th Bit

Use: $\text{number} \wedge (1 \ll (x - 1))$.

Example: For $n = 5$ (101 in binary), to toggle 2nd bit: $5 \wedge (1 \ll 1) = 101 \wedge 010 = 7$.

10. Check if Two Numbers are Equal

Use: $a \wedge b$.

Example: If $a = 5$, $b = 5$, then $a \wedge b = 0$ (Equal).

Advanced Concepts

1. Finding Unique Elements

Given an array where every number appears twice except one, find the unique number using XOR.

Example: $[2, 3, 5, 3, 2]$. XORing all elements gives 5 (the unique number).

2. XOR Property for Swapping Numbers

If $c = a \wedge b$, then XORing any two numbers among a , b , and c will yield the third.

Example: $a = 3$, $b = 5$, $c = 3 \wedge 5 = 6$. $6 \wedge 3 = 5$, $6 \wedge 5 = 3$.

3. XOR Doubly Linked List

A memory-efficient linked list where the next and prev pointers are stored as a single value using XOR.

Example: For nodes A , B , C , where $B = A \wedge C$, retrieving next node is efficient using XOR.

Bitwise Challenges

1. Reset the Right-most SET Bit

Use: $\text{number} \& (\text{number} - 1)$.

Example: For $n = 12$ (1100 in binary), $n \& (n - 1) = 1100 \& 1011 = 1000$.

2. Find the Bit Length of a Number

Increment a counter while right-shifting:

Example:

$n = 5$

```
len = 0
```

```
while n:
```

```
    len += 1
```

```
    n >>= 1
```

Result: len = 3 (for 101 in binary).

3. Count the Number of SET Bits

__builtin_popcount() in STL C++ : Count Set bits

Use a similar loop:

Example:

```
cnt = 0
```

```
while n:
```

```
    cnt += 1
```

```
    n &= (n - 1)
```

Result: For n = 5 (101 in binary), cnt = 2.

4. Find All Subsets of a Set

Loop from 0 to $2^n - 1$ to find all subsets using bitwise operations.

Example:

For set [a, b], subsets are:

0: {}

1: {a}

2: {b}

3: {a, b}

Gosper's Hack

Iterate over subsets of size exactly K

First extract the rightmost 1 bit of the current mask.

Next, set the last non-trailing bit to 0 and clear to the right (carry bit).

Finally, produce a block of 1s at the least-significant bit.

Gosper's Hack Explained

Gosper's Hack* is famous. Written by and named after Ralph William Gosper, Jr., it comes up any time someone on a public forum asks for people to share their favorite programming tricks or hacks.

The Code

Here is Gosper's Hack, written as a C function:

Code

```
void GopersHack(int k, int n)
{
    int set = (1 << k) - 1;
    int limit = (1 << n);
    while (set < limit)
    {
        DoStuff(set);
        // Gosper's hack:
        int c = set & - set;
        int r = set + c;
        set = ((r ^ set) >> 2) / c | r;
    }
}
```

DoStuff() is meant to be replaced with a function that processes each different value that *set* takes.

What It Does

Gosper's Hack iterates through all n -bit values that have k bits set to 1, from lowest to highest. Given $k = 3$ and $n = 6$, it will produce the following values, in order from least to greatest:

000111_b (7_d) 011001_b (25_d) 101010_b (42_d)

001011_b (11_d) 011010_b (26_d) 101100_b (44_d)

001101_b (13_d) 011100_b (28_d) 110001_b (49_d)

001110_b (14_d) 100011_b (35_d) 110010_b (50_d)

010011_b (19_d) 100101_b (37_d) 110100_b (52_d)

010101_b (21_d) 100110_b (38_d) 111000_b (56_d)

010110_b (22_d) 101001_b (41_d)

For any $n \geq k$, Gosper's Hack selects exactly the ${}_nC_k$ values with k out of the first n bits set to 1.

How Would You Do It?

Before we see how Gosper's Hack does it, let's see how we could do it manually. Is there a set of rules that will start with 000111_b, and move to 001011_b, then to 001101_b, and so on?

Starting out is easy: *Set the rightmost k bits to 1.* For $k = 3$, this gets us: 000111_b.

This is exactly what the first line of Gosper's Hack does:

```
int set = (1 << k) - 1;
```

There is also a straightforward rule for moving to the next number:

1. Find the rightmost 1-bit that can be moved left into a 0-bit. Move that 1-bit left one position.
2. Move all 1-bits that are to the right of that bit all the way to the right.

Here are those rules in action:

Set the rightmost k bits to 1

000111_b

00**0**111_b => 00**1**011_b

Find the rightmost 1-bit that can be moved left. Move it left one.

Move all 1-bits that are to the right of that bit all the way to the right. 0010**1**1_b => 0010**1**1_b

Result 001011_b

001**0**11_b => 001**1**01_b

Find the rightmost 1-bit that can be moved left. Move it left one.

Move all 1-bits that are to the right of that bit all the way to the right. 00110**1**_b => 00110**1**_b

Result 001101_b

0011**0**1_b => 0011**1**0_b

Find the rightmost 1-bit that can be moved left. Move it left one.

Move all 1-bits that are to the right of that bit all the way to the right. 001110_b => 001110_b

Result 001110_b

001110_b => **0**10110_b

Find the rightmost 1-bit that can be moved left. Move it left one.

Move all 1-bits that are to the right of that bit all the way to the right. 010**1**10_b => 0100**1**1_b

Result 010011_b

We should stop when moving a 1-bit to the left exceeds the n -bit range given. For $n = 6$, we should stop when we are given 0111000_b and produce 1000011_b. This check is performed by the 2nd and 3rd lines of Gosper's Hack:

```
int limit = (1 << n);
```

```
while (set < limit)
```

How Gosper's Hack Does It

Now that we know what to expect, let's see how Gosper's Hack takes the current value and produces the next one. The first line in the process is this:

```
int c = set & -set;
```

By definition, $-set + set = 0$. If we were to construct $-set$ from set , we would start by putting a 1 in set 's rightmost 1 bit. Adding that to set will trigger the zeroing out of the rightmost cluster of 1's in set :

When $set = 001110_b$
 $-set$ (partial) = 000010_b
 $set + -set$ (partial) = 010000_b

To complete the zeroing out of all bits in set , we need to put a 1 in $-set$ wherever there's a 0 in set , left of the rightmost 1 in set :

When $set = 001110_b$

$-set = 110010_b$

$set + -set = 000000_b$

Note that there is only one position where set and $-set$ both have a 1 – the rightmost 1 in set . This means that $set \& -set$ gives us the rightmost 1 bit in set . This line alone should be added to the list of clever hacks: $x \& -x$ yields the lowest 1-bit in x .

```
int c = set & -set; // c is equal to the rightmost 1-bit in set.
```

The next line adds c to set , which clears out the rightmost cluster of 1-bits in set and puts a 1 in the first zero left of the rightmost cluster of 1 bits:

```
int r = set + c;
```

Given $set = 00110**0**1110_b$

$c = 00000**0**010_b$

$r = 00110**1**0000_b$

This accomplishes step 1 of the manual process outlined above: *Find the rightmost 1-bit that can be moved left into a 0-bit. Move it*

left one. All that is left is to take the other bits of the rightmost cluster of 1 bits and place them as far to the right as possible. The last line of Gosper's Hack does exactly that. Let's look at it, one operation at a time:

$(r \wedge \text{set})$

Xor'ng r and set returns a cluster of 1-bits representing the bits that were changed between set and r :

```
Given set = 0011001110b
      c = 0000000010b
      r = 0011010000b
      r ^ set = 0000011110b
```

This includes the original rightmost cluster of 1-bits from set , plus one more bit. That is, it contains all the bits that need to be moved to the right – plus two more bits.

How do we shift these bits over? Recall that c is the value of the rightmost bit in set , and therefore also the rightmost bit in $r \wedge \text{set}$. Dividing $r \wedge \text{set}$ by c will move all the bits to the right far enough to remove all trailing zeros:

```
Given set = 0011001110b
      c = 0000000010b
      r = 0011010000b
      r ^ set = 0000011110b
      (r ^ set) / c = 0000001111b
```

This still contains two more 1-bits than we need. We can get rid of those by right-shifting the result by two more bits:

```
Given set = 0011001110b
      c = 0000000010b
      r = 0011010000b
      r ^ set = 0000011110b
      (r ^ set) / c = 0000001111b
      ((r ^ set) / c) >> 2 = 0000000011b
```

Gosper chose to use $((r \wedge \text{set}) \gg 2) / c$ instead of $((r \wedge \text{set}) / c) \gg 2$. Why? I believe he did so because some CPUs use the shift-and-subtract method to perform division and stop as soon as the remainder is 0, so dividing by a small number is faster. Thus, in some CPUs, we can expect $((r \wedge \text{set}) \gg 2) / c$ to be marginally faster than $((r \wedge \text{set}) / c) \gg 2$.

All that is left is to combine the right-shifted bits with r :

$\text{set} = (((r \wedge \text{set}) \gg 2) / c) | r;$

```
Given set = 0011001110b
      c = 0000000010b
      r = 0011010000b
      r ^ set = 0000011110b
      (r ^ set) / c = 0000001111b
      ((r ^ set) >> 2) / c = 0000000011b
      set = (((r ^ set) >> 2) / c) | r = 0011010011b
```

With that final step, Gosper's Hack accomplishes the iteration from one integer with k bits set to the next higher integer with k bits set. The final check stops it when more than n bits are used.

That is the entirety of Gosper's Hack. To see it run on a full sample, from start to finish, skip the next section.

Iterate over subsets of sizes $\leq K$

Lets iterate over all the subsets of size X , $0 \leq X \leq K$. The approach here is very similar to the one before, we simply have to modify our way of how to determine the next mask in each loop.

Code

```
int count1(int n)
{
    int cnt = 0;
    while (n)
    {
        cnt++;
        n &= (n - 1);
    }
    return cnt;
}

void DoStuff(int mask)
{
    std::bitset<8> y(mask);
    std::cout << y << '\n';
}

void Subset_leq_k(int k, int n)
{
    int mask = 0;
    int limit = (1 << n);
    int cc = 1;
    while (mask < limit)
    {
        cout << cc++ << ": ";
        DoStuff(mask);
        if (count1(mask) < k)
            mask++;
        else
        {
            int c = mask & -mask;
            mask = mask + c;
        }
    }
}
```

In order to determine the next mask, we check whether the current mask has less than K 1 bits set. In that case, we know that by adding 1 at maximum one more bit will be set to 1. Otherwise we determine the mask's lowest set bit and add it to its value.

For example: 00100011100 , in order to maintain at most 4 1s

We add 100 to it

00100011100 + 100 = 00100100000

In this case we got

$$\sum_{i=0}^K \binom{N}{i}$$

subsets.

bitset and its application

A bitset is an array of bool but each Boolean value is not stored separately instead bitset optimizes the space such that each bool takes 1 bit space only, so **space taken by bitset bs is less than that of bool bs[N] and vector bs(N)**. However, a limitation of bitset is, **N must be known at compile time, i.e., a constant** (this limitation is not there with vector and dynamic array)

As bitset stores the same information in compressed manner the operation on bitset are faster than that of array and vector. We can access each bit of bitset individually with help of array indexing operator [] that is bs[3] shows bit at index 3 of bitset bs just like a simple array. Remember bitset starts its indexing backward that is for 10110, 0 are at 0th and 3rd indices whereas 1 are at 1st 2nd and 4th indices.

We can construct a bitset using integer number as well as binary string via constructors which is shown in below code. The size of bitset is fixed at compile time that is, it can't be changed at runtime.

The **main function defined for bitset class** are operator [], count, size, set, reset and many more they are explained in below code –

Code

```
// C++ program to demonstrate various functionality of bitset
#include <bits/stdc++.h>
using namespace std;

#define M 32

int main()
{
    // default constructor initializes with all bits 0
    bitset<M> bset1;

    // bset2 is initialized with bits of 20
    bitset<M> bset2(20);

    // bset3 is initialized with bits of specified binary string
    bitset<M> bset3(string("1100"));

    // cout prints exact bits representation of bitset
    cout << bset1 << endl; // 00000000000000000000000000000000
    cout << bset2 << endl; // 000000000000000000000000000010100
    cout << bset3 << endl; // 00000000000000000000000000001100
    cout << endl;

    // declaring set8 with capacity of 8 bits

    bitset<8> set8; // 00000000

    // setting first bit (or 6th index)
    set8[1] = 1; // 00000010
    set8[4] = set8[1]; // 00010010
    cout << set8 << endl;

    // count function returns number of set bits in bitset
    int numberof1 = set8.count();

    // size function returns total number of bits in bitset
    // so there difference will give us number of unset(0)
    // bits in bitset
    int numberof0 = set8.size() - numberof1;

    cout << set8 << " has " << numberof1 << " ones and "
         << numberof0 << " zeros\n";

    // test function return 1 if bit is set else returns 0
```

```

cout << "bool representation of " << set8 << " : ";
for (int i = 0; i < set8.size(); i++)
    cout << set8.test(i) << " ";

cout << endl;

// any function returns true, if at least 1 bit
// is set
if (!set8.any())
    cout << "set8 has no bit set.\n";

if (!bset1.any())
    cout << "bset1 has no bit set.\n";

// none function returns true, if none of the bit
// is set
if (!bset1.none())
    cout << "bset1 has some bit set\n";

// bset.set() sets all bits
cout << set8.set() << endl;

// bset.set(pos, b) makes bset[pos] = b
cout << set8.set(4, 0) << endl;

// bset.set(pos) makes bset[pos] = 1 i.e. default
// is 1
cout << set8.set(4) << endl;

// reset function makes all bits 0
cout << set8.reset(2) << endl;
cout << set8.reset() << endl;

// flip function flips all bits i.e. 1 <-> 0
// and 0 <-> 1
cout << set8.flip(2) << endl;
cout << set8.flip() << endl;

// Converting decimal number to binary by using bitset
int num = 100;
cout << "\nDecimal number: " << num
    << " Binary equivalent: " << bitset<8>(num);

return 0;
}

```

Output :

```

00000000000000000000000000000000
00000000000000000000000000000010100
00000000000000000000000000000001100
00010010
00010010 has 2 ones and 6 zeros
bool representation of 00010010 : 0 1 0 0 1 0 0
bset1 has no bit set.
11111111
11101111
11111111
11111011
00000000
00000100
11111011

```

Decimal number: 100 Binary equivalent: 01100100

For bitset set, reset and flip function are defined. Set function sets (1) all bits of bitset if no argument is provided otherwise it sets the bit whose position is given as argument. In same way reset and flip also work if they are called with no argument they perform their operation on whole bitset and if some position is provided as argument then they perform operation at that position only.

For bitset all bitwise operator are overloaded that is they can be applied to bitset directly without any casting or conversion, main overloaded operator are &, |, ==, != and shifting operator <> which makes operation on bitset easy.

Use of above operator is shown in below code.

Code

```
// C++ program to show applicable operator on bitset.
#include <bits/stdc++.h>
using namespace std;

int main()
{
    bitset<4> bset1(9); // bset1 contains 1001
    bitset<4> bset2(3); // bset2 contains 0011

    // comparison operator
    cout << (bset1 == bset2) << endl; // false 0
    cout << (bset1 != bset2) << endl; // true 1

    // bitwise operation and assignment
    cout << (bset1 ^= bset2) << endl; // 1010
    cout << (bset1 &= bset2) << endl; // 0010
    cout << (bset1 |= bset2) << endl; // 0011

    // left and right shifting
    cout << (bset1 <<= 2) << endl; // 1100
    cout << (bset1 >>= 1) << endl; // 0110

    // not operator
    cout << (~bset2) << endl; // 1100

    // bitwise operator
    cout << (bset1 & bset2) << endl; // 0010
    cout << (bset1 | bset2) << endl; // 0111
    cout << (bset1 ^ bset2) << endl; // 0101
}
```

Output :

```
0
1
1010
0010
0011
1100
0110
1100
0010
0111
0101
```